SPEX / ARCHITECTURE SERIES

SpeX dApp Protocol

Interoperable runtime, agent-native interfaces, portable proof and selective chain authority

WORKING CONCEPT v0.4

ANGI / spex.json + spex.md

OpenSpeX Receipts / Nostr / ROD

OmniSpatialGraph: future integration boundary

23 October 2026

Conceptual architecture | Not frozen

WORKING CONCEPT / 23 OCTOBER 2026

Executive definition

SpeX dApp Protocol is a lightweight interoperability contract for digital applications and autonomous environments. It enables an application to expose its live operational state and contextual knowledge through ANGI, exchange signed events over Nostr, produce portable OpenSpeX Receipts for verifiable outcomes, and use ROD selectively for authority, ownership, and settlement.

> **Governing design**
> One application runtime. One agent-native interface. Two complementary outputs. One portable outcome receipt. Optional blockchain operations.

The protocol is designed for minimum viable compatibility, not mandatory adoption of a complex verification or graph infrastructure. Simple applications can remain simple. More demanding environments can activate stronger controls without changing the external contract.

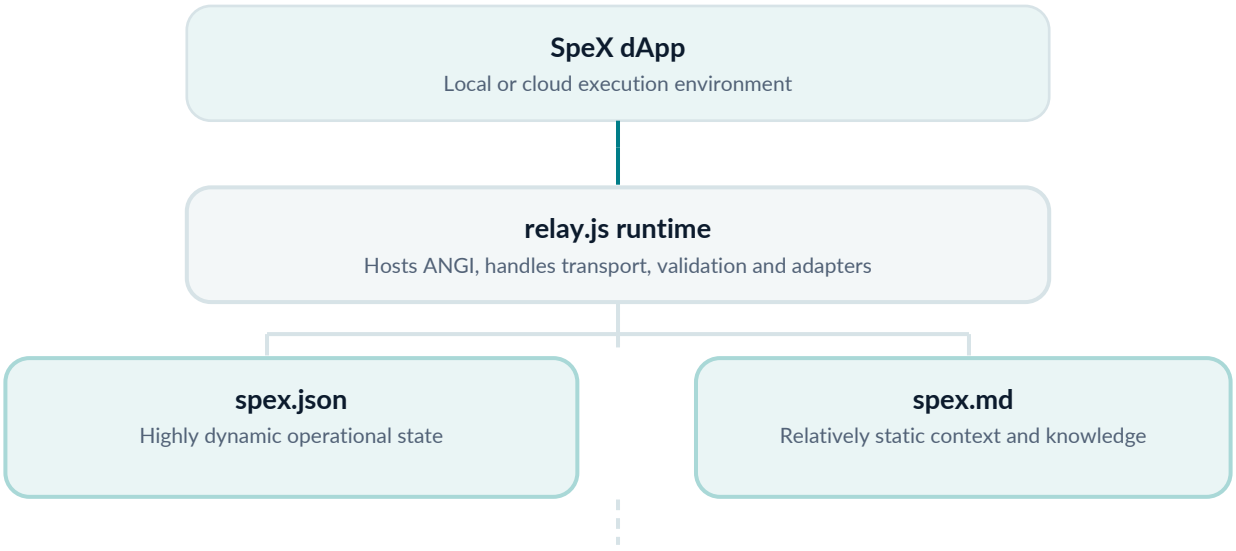
Invariant	Protocol position
ANGI	Emits both **spex.json** and **spex.md**.
spex.json	Highly dynamic, machine-readable operational state.
spex.md	Relatively static, agent-readable context and semantics.
relay.js	Hosts ANGI outputs and coordinates the dApp; it does not independently author either output.
OmniSpatialGraph	Future graph-native provider; not mandatory for current conformity.
Identity	One user-controlled ROD/Nostr private key is the intended common cryptographic identity.
Verification	Minimum common contract for claims, checks, signed receipts, and declared assurance.
ROD	Authoritative names, selected commitments, ownership and settlement; not a realtime event log.

Intended outcome

A dApp can expose a consistent state-and-action surface to agents, create auditable receipts of meaningful work, and retain deployment freedom. Users should not be required to understand blockchain operations during ordinary interaction. The same conceptual protocol applies to local software, hosted services, spatial environments, and autonomous applications.

1 Scope and architectural boundaries

SpeX dApp Protocol defines interfaces and responsibilities between an application, its relay runtime, agent clients, evidence validators, Nostr transport, and optional ROD authority or settlement. It does not define a universal gameplay model, universal service marketplace, general-purpose blockchain, or full graph database.



ANGI produces both views; OSG may supply richer graph data later.

Application logic remains authoritative for its domain. The dApp defines its own rules, controls its real execution state, and authorizes any state-changing operation. ANGI makes selected information available to agents; ANGI does not replace the rules engine or grant permission merely by describing an action.

Separation of concerns. The application determines state. ANGI expresses permitted views. Nostr transports events. An application-specific verifier judges a claim. OpenSpeX defines the portable receipt. ROD establishes chosen public authority and settlement. OSG can later supply versioned graph structure behind ANGI.

2 ANGI: one producer, two information timescales

Agent Native Graph Interface (ANGI) is the common agent-facing interface. It emits two complementary representations: a frequently changing structured operational view and a more durable explanatory knowledge view. Both are derived from authorized application information; neither is an independent database.

Output	spex.json - dynamic	spex.md - contextual
Meaning	What the application knows and permits *now*.	What the application is, what its concepts mean, and how its rules should be interpreted.

Output	spex.json - dynamic	spex.md - contextual
Typical fields	Current state, revision, offered actions, action constraints, task status, scopes, capability availability, references.	Purpose, operating semantics, policies, architecture, terminology, stable rules and applicable context.
Lifecycle	Regenerate or compute on demand when relevant state changes; avoid unnecessary disk writes.	Cache until underlying contextual knowledge changes; explicitly version its content.
Audience	Agent software, dApp clients and protocol integrations.	Reasoning systems, developers, auditors and authorized agents.
Producer	ANGI.	ANGI.

A synchronized contract

Every dynamic response should identify its **state revision** and the **context digest/version** used to interpret it. Operations that depend on a state snapshot should carry the expected revision. The runtime compares it with the actual current state before committing the action; stale or unauthorized requests are rejected or re-evaluated.

Context versions must remain retrievable where needed to interpret issued receipts. This does not require retaining private or personal information indefinitely. Public, stable context can be archived separately from revocable private session data.

Readability and confidentiality

ANGI emits the **view authorized for the requester**, not a universal dump of application memory. Public discovery, a logged-in participant, a validator and a privileged operator can receive different subsets. Capabilities advertised in either file are descriptive; all execution permissions are independently enforced by the runtime.

The protocol may offer an action endpoint, but its exact URL and method are an implementation contract to be specified, not a fixed property of the ANGI file formats. Stable JSON schema versions and output content revisions must remain separate.

3 Reference runtime: relay.js

relay.js is the reference dApp coordinator and hosting surface. It serves the ANGI outputs, receives signed submissions, executes application-specific validation, issues authorized receipts, communicates through Nostr, and interacts with ROD or storage only through configured capabilities. It is not the canonical producer of **spex.json** or **spex.md**: ANGI is.

The runtime should discover only configured capabilities and registered modules. It must never announce a verifier, agent action, payment ability, or graph service that is not operational. A missing feature is advertised as unavailable rather than simulated.

Logical runtime responsibilities

- Host ANGI representations and their revision metadata through stable discovery endpoints.

- Authenticate submitted events, perform idempotency checks, enforce authorization, and reject stale state-changing commands.
- Apply domain-specific validation to claims and issue signed OpenSpeX Receipts where a verifier is configured.
- Route optional Nostr events, encrypted communication, and exactly identified evidence references; treat external relays as untrusted.
- Expose a tightly restricted application-owned ROD transaction capability where configured; never custody the player's private key.
- Persist required operational records and recovery state using deployment-appropriate adapters.

Local and cloud execution

The intended local distribution is a Node.js **Single Executable Application (SEA)** or equivalent self-contained build. The executable includes the Node.js runtime and bundled JavaScript dependencies, so end users do not separately install Node.js or run npm. Native addons, writable data, certificates, platform libraries, and access to ROD Core still require deliberate packaging or configuration.

A cloud deployment can expose the same external contract through a worker or service runtime. A cloud worker is not expected to bundle a native database or long-running Node.js server. Storage, signer and communication adapters vary; the manifest and receipt semantics do not.

4 Identity, key continuity, and privacy

The intended common user identity is one user-controlled **secp256k1 private key** usable with the corresponding ROD and Nostr cryptographic operations. Matching underlying key material does not mean their public-key encodings or signing conventions are identical: the protocol must define conversion and proof-of-control checks precisely. The application has a separate, application-owned issuer or transaction key.

Initial interaction should not require a wallet or exposed blockchain ceremony. Persistent signed participation may activate the existing user identity as needed; value transfer requires user control of the receiving address. The coordinator must never extract, retain or operate the participant's private spending key.

Key destruction and unlinkability

A single key can intentionally be the root of access to private identity mappings and encrypted evidence. When no recovery copy or independent linking record remains, destroying the effective decryption secret can make that protected linkage unrecoverable. Cryptographic domain separation, authenticated encryption, minimal metadata, and deletion of plaintext caches are required to make this meaningful.

> **Important privacy limit**
> Key destruction does not remove immutable ROD records, erase previously published Nostr events, invalidate historical signatures, or delete information held by third parties. A stable public key, public receipt link, or deterministic hash may continue to identify or correlate a person. Key destruction alone is not a general GDPR erasure mechanism.

The preferred structure separates **public outcome commitments** from **private identity-to-evidence mappings**. Receipts can retain independently verifiable integrity while contextual access becomes intentionally revocable.

Privacy obligations still require minimization, retention controls, access management, and legal assessment of publicly accessible records.

A lost single root key may also irreversibly lose access to owned ROD value. This is a consequence to communicate, not an issue the protocol can silently eliminate.

5 Events and messaging over Nostr

Nostr is the fast distribution and communication layer. Standard event signing and ID calculation support authenticated messages; relays transport those messages but do not establish correctness or authorization of a task result. The application verifies event signatures, payload structure, replay rules, source authority and requested operations independently.

Transport behavior adopted from node architecture patterns

- **Exact-ID recovery:** fetch missing referenced objects by event ID, preferably through more than one relay.
- **Bounded resilience:** cap queues, retries, subscription sizes, payloads and expensive-verification work.
- **Safe relay defaults:** configurable read/write relays; an embedded restricted relay is optional, not a mandatory public service.
- **Deterministic reconstruction:** replay applicable events against pinned rules where the application can reproduce the state transition.
- **Explicit trust boundary:** transport acknowledgments mean delivery acceptance, not proof validation or ROD settlement.

Confidential transport can use the relevant Nostr authenticated-encryption standard, with optional metadata-protection layers where supported. Signed public posts must not be misrepresented as confidential. Sensitive evidence should be stored under a defined retention and access policy, rather than assumed erasable merely because it passed through a Nostr relay.

6 Verification and the portable receipt

The initial dApp protocol targets **minimum viable verification compatibility**. It does not impose a universal proof method. Instead, it provides a common lifecycle and a signed, portable record of the decision reached under a declared method.

Stage	Required meaning
Claim	A named action or task result is submitted under an identified application/rules context.
Validation	A declared verification process assesses the relevant evidence or policy conditions.
Receipt	An authorized issuer signs the result, preserving the method, context and evidence references.
Optional anchor	A ROD transaction may commit to a verified receipt or batch.
Optional settlement	A distinct value or ownership transfer is executed and confirmed on ROD.

OpenSpeX Receipt: minimum semantic fields

A conforming receipt should capture **receipt version; issuer; application/task identity; claimed outcome; verification status and method; applicable context digest; state revision or state-commitment reference; permitted evidence commitment; issuance time or logical sequence; and a verifiable signature**. A receipt may refer to earlier receipts without requiring that all details be exposed publicly.

Receipt signatures require an unambiguous canonical byte format, signing domain, issuer key representation, and validation procedure. Hashing loosely serialized JSON is insufficient for interoperable cross-implementation verification.

Assurance must be honest: a signature proves origin, not truth; a valid evidence hash proves matching bytes, not task correctness; and a confirmed blockchain transaction proves settlement or commitment, not necessarily a valid underlying outcome. A reviewer may require disclosed evidence or trust an authorized verifier when the evidence remains private.

Privacy-preserving dual views

The application may retain a detailed restricted receipt and issue a minimized public receipt. The public representation should include no unnecessary stable participant key, private graph locator, raw trace, or low-entropy digest that permits reconstruction or cross-service correlation. The two views must bind to the same outcome through an appropriately designed commitment.

7 ROD: authority and economic finality

ROD remains the authoritative chain for app-controlled names, digital ownership, chosen public commitments, and settlement. Real-time operational state does not belong on-chain by default. Ordinary ANGI state refreshes, navigation, and private evidence updates need no blockchain transaction.

A dApp-owned name can authenticate the application issuer and, where useful, reference a manifest signature or periodically committed result root. Nostr then distributes live events and signed receipts. Applications choose when a commitment merits a fee and public permanence.

Selective anchoring

Receipts may be batched into a deterministically computed commitment, reducing repeated transactions. A commitment supports later inclusion proofs where the protocol defines them, but it does not by itself prove that every eligible claim was included or that validation was fair. Such fairness properties require additional auditability.

Financial independence

An application may sponsor operational ROD fees from an application-controlled wallet. Participant-owned funds and assets must not be represented as noncustodial until they are controlled by the participant's key. Rewards, vouchers, ownership changes and final on-chain settlement are distinct events with separate conditions and receipts.

8 Storage and graph evolution

Storage is an adapter behind the contract. A local runtime can use a local embedded database; a hosted runtime can use a managed data service. This choice must not alter the ANGI formats, receipt validation, identity semantics, or public authority rules. Private evidence should remain outside irreversible ledgers unless a narrow, justified use requires otherwise.

ANGI works before full OmniSpatialGraph generalization

In the first stage, ANGI builds its permitted views from the application's native state, declarative rules, and configured knowledge sources. The interface must therefore not require OSG as a runtime dependency.

When OmniSpatialGraph supports generalized application state, ANGI can project from its versioned entities, relationships, decisions, provenance and history. OSG remains the source of graph knowledge; ANGI remains the public emitter of both **spex.json** and **spex.md**. Future OSG implementations may generate the underlying contextual material but should not introduce a competing externally authoritative manifest.

OSG's present repository-memory prototype already demonstrates content-addressed graph history, but full spatiotemporal state, signed receipts and production game/service workloads are future extensions. The protocol treats those as an integration direction, not a shipped prerequisite.

9 Compatibility profiles and OIP alignment

Profile	Conforming capability	Additional dependencies
Basic dApp	ANGI emits dynamic JSON and relatively static Markdown; relay serves both with correct versions and scopes.	No wallet, public relay, receipt issuer, or OSG runtime required.
Verified dApp	Adds authenticated submission, application-defined verification, and signed OpenSpeX Receipts.	Configured verifier and protected evidence handling.
ROD-enabled dApp	Adds authorized app names, ownership, anchoring, rewards, payments or settlement.	ROD client/signer and declared confirmation policy.
OSG-enhanced dApp (future)	Uses generalized graph-native history/provenance as ANGI input.	Production-ready OSG application adapter.

These profiles extend a single protocol. Applications advertise only actual capabilities. Profiles can be combined without imposing higher-tier functions on lower-tier implementations.

Existing OpenSpeX standards family

OIP	Role in the SpeX dApp Protocol
OIP-0000	Standards process and compatibility governance.
OIP-0001	spex.json: dynamic ANGI output and discovery; references the spex.md contextual output.
OIP-0002	OpenSpeX Receipt: portable, signed, evidence-backed outcome artifact.
OIP-0003	Reputation: optional interpretation of verified history.
OIP-0004	ROD integration: identity, ownership, authority, commitments and settlement.

The separate OSEP evidence-packet proposal is deprecated and should not be reintroduced. The receipt is the portable evidence artifact. The details of spex.md versioning remain an ANGI/OIP-0001 design item rather than justification for a second competing protocol stack.

10 Security and lifecycle guarantees

The protocol should enforce authorization at the runtime, not rely on agent-readable descriptions, encrypted transport alone, or friendly relay behavior. Operational state may be dynamic, but permission checks must run against the authoritative current state.

- **Trust:** treat every relay event and user-supplied document as untrusted data until authenticated and validated; never promote graph content into executable agent instructions.
- **Concurrency:** carry state revisions, reject incompatible stale actions, deduplicate retries, and maintain safe operation ordering.
- **Proof:** pin rule/context digests for old receipts; define canonical signing; support independent checks only to the declared assurance level.
- **Privacy:** minimize public identity linkage; apply scoped ANGI projections, access controls, data retention and disposal policies.
- **Keys:** protect user-controlled root secrets; isolate application signing authority; place spending bounds on relay capabilities.
- **Availability:** bound queues and subscriptions, persist critical recovery state, and avoid making a single external relay the sole evidence store.
- **Deployment:** bind local administrative endpoints conservatively, protect signer material, and document external node dependencies.

Non-goals of the initial protocol

The first compatibility version does not require zero-knowledge proofs, a generalized OSG database, universal replay of nondeterministic workloads, automated stake-slashing, decentralized consensus on every task, a new blockchain, a public Nostr relay, a full wallet, or a universal reputation score. Each could be explored later without bloating the base contract.

11 Decision record and open work

Established direction	Future formalization needed
ANGI emits both files; JSON dynamic, Markdown contextual.	Field schema, representation authentication, scope filters, update and version rules.
relay.js hosts the emitted views.	Stable endpoints, deployment adapters, failure handling and conformance tests.
One user-controlled ROD/Nostr key is intentional.	Key encoding, signature-domain rules, encryption-linkage and loss/recovery disclosure.
Portable signed OpenSpeX Receipts; verification can be minimal.	Canonical receipt bytes, issuer authority, assurance vocabulary, evidence policy.
ROD anchors selectively and settles value.	On-chain naming/commitment formats, reorganization policy and sign limits.
Generalized OSG integration is future work.	Graph projection contracts and migration with no break to existing ANGI clients.

Recommended adoption criterion

A conforming basic implementation can emit and serve valid ANGI views with coherent versions and access scope. A conforming verified implementation can authenticate a result and issue a verifiable receipt. Optional ROD and OSG capabilities do not change either baseline guarantee.

12 Reference basis and document status

This text synthesizes the current SpeX / OpenSpeX working decisions and published technical specifications. It is a **proposed architecture**, not a claim that all named interfaces or integrations exist today. Generic protocol requirements are distinguished from implementation commitments. No application-specific case studies form part of this document.

Primary reference materials

OmniSpatialGraph - README and MVP specification
https://github.com/bellodox/OmniSpatialGraph

Nostr Blockchain - node architecture, branch new
https://github.com/Nostr-Blockchain/Node/tree/new

SpeXex - ROD/Nostr cryptographic interoperability
https://github.com/SpeXexHQ/SpeXex

Nostr protocol - NIP-01 signed events
https://github.com/nostr-protocol/nips/blob/master/01.md

Nostr protocol - NIP-44 versioned encryption
https://github.com/nostr-protocol/nips/blob/master/44.md

Nostr protocol - NIP-59 gift wrapping
https://github.com/nostr-protocol/nips/blob/master/59.md

Node.js - Single Executable Applications
https://nodejs.org/api/single-executable-applications.html

European Data Protection Board - Guidelines 02/2025, blockchain processing
https://www.edpb.europa.eu/our-work-tools/our-documents/guidelines/guidelines-022025-processing-personal-data-through_en

Working status: Concept v0.4, dated 23 October 2026. Not an accepted OIP, shipped reference implementation, legal opinion, or compatibility certification. The protocol name and core roles are proposed here for review and future formalization.